CIUK Cluster Challenge 2026: Durham subchallenge

Team plan, Tue 20 to Sun 25 October

Team: TODO-team-name Users: TODO ×4 Repo: TODO-link

At a glance

- **Job days:** Tue 20, Wed 21, Thu 22, and Fri 23 until the queues close at **16:00**. Submit final reruns by **Fri 12:00**.
- **Write-up:** Fri afternoon, Sat 24 and Sun 25. The report is **2 pages A4 maximum** and goes with a presentation.
- **Access:** `login8.cosma.dur.ac.uk`, account `do022`, queues `ciuk-dine` and `ciuk-dine2`. Jobs run for at most **1 hour** and must use exclusive nodes.
- **Nodes:** DINE is **2× AMD EPYC 7302** (Zen 2, 16 cores each, 512 GB), per the COSMA docs. DINE2's CPU is **not documented**, so the recon job on Tuesday morning finds it out.
- **Code changes are out of scope.** Compiler and runtime settings are allowed, but document every flag.
- **Every submitted item** must list the team name and all four usernames.

1 Where the marks are, and what that means for us

Marked item	Pts	What we do about it
likwid-bench findings, node 1 / node 2	20 + 20	Owners: B (DINE) and C (DINE2). Same script on both nodes. Sweep thread count, placement and working-set size, and compare against theoretical peaks.
Node 1 vs node 2 evaluation	10	Owner: C. One table with both nodes' peaks, measured values, efficiency and ridge point.
Grid behaviour, node 1 / node 2	10 + 10	Owner: D. Strong scaling for each L on each node. Explain where it flattens using the likwid data.
Grid vs LIKWID	10	Owner: D. Roofline with measured roofs, plus Grid's point from likwid-perfctr.
Best-case Grid throughput	10	Owner: A. Separate tuning runs over compiler, binding, NUMA and L. Report the best number and its exact config.
Time efficiency of testbed use	10	Everyone. Job arrays, an overnight queue that is always full, `runlog.csv`, and the job count in the report.

Half the marks come from likwid-bench (50 of 100). B and C own that work full time, and D and A shouldn't take queue slots from it on Tuesday.

2 Who does what

Person	Job days (Tue to Fri)	Write-up (Fri to Sun)
A: build and tuning lead, editor	Builds Grid for both node types and a second compiler. Owns `config/`, `build/` and the repo rules. After the builds, runs the best-case throughput tuning.	Setup section (hardware table, software stack, all flags, experiment count), conclusions, page budget, final edit, slide deck.
B: DINE likwid owner	DINE recon, module fixes, theoretical peaks, all likwid-bench runs on DINE. Reviews D's Grid data on DINE for consistency.	§2.1 DINE capability: bandwidth and FLOP/s figures, about 0.35 pages.
C: DINE2 likwid owner	Same as B for DINE2. Finds out DINE2's CPU **by Tue 11:00** and tells A which SIMD build to make.	§2.2 DINE2 and §2.3 node comparison table, about 0.45 pages.
D: Grid and analysis lead	`analysis/` pipeline, Grid strong scaling on both nodes, likwid-perfctr on Grid, roofline. Checks that perfctr works on day 1.	§3 Grid and §4 Grid vs LIKWID and roofline, about 0.9 pages, plus every figure.

Pairing for cover: A↔D (Grid) and B↔C (likwid). If someone's account isn't active, their pair partner submits their jobs while they do analysis and reading.
Syncs: 15 minutes at **09:30** (what's in the queue, what failed) and **18:00** (look at plots, agree the overnight queue).

Someone checks the queue last thing at night.

3 Day-by-day schedule

Tue 20: recon, builds, smoke tests, first overnight queue

All	09:00: clone the repo on COSMA, fill in `config/team.env`, and check everyone can submit.
B	09:15 `scripts/submit.sh dine recon`. Fix modules in `config/dine.env`. Fill in `analysis/peaks.py` from the spec sheet, noting the DIMM speed and NUMA (NPS) mode from `likwid-topology`. Smoke test `likwid_bench -- threads`, check the CSV, then queue repeats.
C	09:15 `scripts/submit.sh dine2 recon`. Identify the CPU, SIMD level (AVX-512?), sockets, NUMA, memory and SMT. **Tell A the SIMD build by 11:00**. Then do the same as B for DINE2.
A	Build `cpu-avx2` for DINE on the login node (`build/build_grid.sh dine cpu-avx2 8`), then the DINE2 build. Submit one `grid --array=32` smoke run on each node and **time it**. The time sets the size of the Grid campaign.
D	Check `perfctr-smoke.txt` from recon. **If counters can't be read, ask on the official CIUK channel straight away.** Run `collect.py` and `plots.py` on the first likwid output. Work out Grid's working set for each L vs cache sizes (see §6) so we have predictions before the data arrives.
18:00	Queue overnight: likwid `threads` ×3 and `sizes` on both nodes, plus Grid strong scaling on DINE if the build is ready (`scripts/campaign.sh dine 3`).

Wed 21: main campaign

A	Finish any build still pending. Build the second compiler (`cpu-avx2-clang`, or AOCC if COSMA has it). First tuning runs: `OMP_PROC_BIND=spread` vs `close`, `NUMACTL=--interleave=all`.
B	Analyse the DINE runs: where bandwidth saturates (cores per NUMA domain and per socket), FLOP/s vs peak, compact vs scatter. Fill gaps (NT-store kernels, a single NUMA domain run). Start bullet notes for §2.1.
C	The same for DINE2. Start the comparison table.
D	Grid strong scaling on DINE2. `MAX_L=48` at 8, 16 and full threads on both nodes. likwid-perfctr runs (`PERFCTR="MEM FLOPS_SP"`) at 1, 8, 16 and full threads, plus one `TIMELINE=1` run at full threads.
18:00	**First full set of figures reviewed.** Write the list of gaps and queue them overnight.

Thu 22: fill the gaps, best-case throughput, weak scaling

A	Best-case sweep: compiler × binding × NUMA policy × SMT on/off, at the best L. Report the best `Gflops_dwf4` with its full config and ≥3 repeats.
B/C	Extra runs Wednesday showed are needed (SMT, frequency behaviour, odd outliers). Draft the section text.
D	Weak scaling: compare runs where L^4 per thread stays roughly constant. Grid's axpy vs likwid triad. Draft roofline figures for both nodes.
18:00	**Freeze.** Agree the final rerun list for Friday morning. Anything not on the list doesn't run.

Fri 23: last reruns, then write-up begins

- **Morning:** Submit reruns from the freeze list by **12:00**. Jobs submitted before 16:00 still run after the queues close. Everyone turns their bullet notes into prose.
- **Afternoon:** D regenerates all figures from final data by 18:00. A assembles draft v0 in `report/report.tex` by end of day.

Sat 24: complete draft and slides

- **Morning:** Complete draft v1. It'll be too long, and that's expected.
- **Afternoon:** A cuts it to 2 pages with the page budget in §5. Each owner makes 2–3 slides for their section, and A builds the deck.

Sun 25: review and submit

- **Cross-review:** B reviews D's sections, D reviews C's, C reviews A's, and A reviews B's. Each reviewer checks every claim against a figure or number.
- **Final checks:** team name and usernames on everything, and the 2-page limit.

- **Rehearse** the presentation, then submit.

4 Experiment matrix

Run priority-1 items first. Only move to the next priority when everything above it is in the queue. The job counts are for both nodes together.

Prio	Experiment	Submit with	Jobs
1	Recon on each node	`submit.sh <n> recon`	2
1	likwid-bench vs threads, compact + scatter, all flop and bandwidth kernels, 3 repeats inside each job, 3 jobs	`likwid_bench -- threads`	6
1	likwid-bench vs working-set size, 1 thread and all cores	`likwid_bench -- sizes`	2
1	Grid strong scaling, $L \leq 32$ (covers 24 and 32), threads 1,2,4,6,8,12,16,20,24,28,32, 3 repeats	`grid --array=...`	66
1	likwid-perfctr on Grid (MEM, FLOPS_SP) at 1, 8, 16 and all threads	`PERFCTR=... TAG=perfctr`	8
2	$L = 48$ at 8, 16 and all threads	`MAX_L=48 TAG=L48`	6
2	Best-case tuning: compiler, binding, `numactl`, SMT	`TAG=tune-...`	12–20
2	perfctr timeline (separate setup from the Dirac phases)	`TIMELINE=1`	2
3	likwid extras: single NUMA domain, SMT, non-temporal stores, repeats at different times of day	`KERNELS=...`	4–8

Around 110 jobs in total. If Tuesday's timing run shows that 1-thread Grid at $L \leq 32$ won't finish in 55 minutes, use `MAX_L=24` for $T \leq 2$. In that case, normalise efficiency to the smallest thread count that finished, and say so in the report.

5 Report plan (2 pages A4, two columns)

Section	Owner	Content and figures	Space
1 Setup	A	Hardware table (theoretical peaks with sources), compilers and flags, pinning policy, number of experiments	0.15p
2.1 DINE	B	`likwid_bw_threads`, `likwid_flops_threads`, `likwid_bw_sizes`: % of peak, saturation point, why	0.35p
2.2/2.3 DINE2, comparison	C	Same figures and a combined table with ridge points	0.45p
3 Grid	D	`grid_scaling_<node>`, `grid_vs_L`, best-case number	0.5p
4 Grid vs LIKWID	D	`roofline_<node>`: Grid-reported vs counter-measured FLOP/s, axpy vs triad	0.4p
5 Conclusions	A	Answer the four questions in the brief in one line each	0.15p

The brief asks four questions, and the report must answer each one explicitly:

1. How does Grid compare to LIKWID, where do they differ, and why?
2. Where does Grid sit on the roofline?
3. How well does Grid strong scale, and can the likwid data explain it?
4. What can we say about weak scaling?

6 Technical notes: hypotheses to confirm or overturn

Theoretical peaks (DINE; fill in DINE2 after recon)

- **Compute:** 2 × 16 cores × 3.0 GHz base × 16 DP FLOP/cycle gives 1.54 TFLOP/s DP and 3.07 TFLOP/s SP. The count of 16 comes from 2 FMA pipes × 4 doubles × 2. At 3.3 GHz boost the figures are about 10% higher. Report both, because all-core AVX clocks sit somewhere in between.
- **Memory:** 2 sockets × 8 channels × DDR4-3200 × 8 B gives 409.6 GB/s. Check the DIMM speed with recon. Expect triad to reach roughly 60–80% of this.
- **Ridge point:** about 3 TFLOP/s SP divided by about 300 GB/s measured, giving roughly 10 FLOP/byte. That's far to the right of where Grid is likely to sit.

likwid-bench pitfalls

- Kernels without NT stores (`copy`, `triad`) trigger **write-allocate**, so likwid's MB/s understates the real traffic. Compare them with the `_mem` (non-temporal) variants and explain the gap.
- The scalar `peakflops` kernel can't reach peak. Only the `_avx_fma` kernel (or AVX-512 on DINE2, if present) can. Plot all of them; the ratios are themselves a finding.
- On Zen 2, L3 is 16 MB per CCX, not shared across the socket. The `sizes` sweep should show this. NUMA layout depends on the NPS mode: read it from `likwid-topology`.

Grid

- **One run covers every L up to `--max-L`.** So $L = 24$ and 32 both come from each run. The JSON's `Gflops_dwf4` and `Gflops_wilson` are in **single precision** (FP64 is switched off by the required flags), so compare them against the **SP** roof.
- **Rough arithmetic intensity, assuming no cache reuse.** The Wilson dslash does about 1320 FLOP per site and moves about 1440 B per site in SP (8 neighbour spinors at 96 B, 8 links at 72 B, and a 96 B output). That gives roughly 0.9 FLOP/B. DWF reuses each link across the fifth dimension, so its AI is higher. **Prediction: memory-bound**, with strong scaling flattening where likwid triad saturates.
- **Working set.** $L = 24$ is 332k sites: about 32 MB per SP fermion field and about 96 MB of gauge field. $L = 32$ is 3.2× larger. Compare these with 2×128 MB of L3 split into 16 MB slices. A drop in GFLOP/s from $L = 24$ to 32 at full threads would be the cache-fit effect.
- **Grid vs counters.** Grid reports *nominal* FLOPs from a fixed formula, while perfctr counts *retired* FP operations for the whole process, setup included. The timeline mode helps separate the two.
- Benchmark_Grid also runs an **axpy memory benchmark**. That gives a direct, low-cost comparison with likwid `stream` and `triad`.

7 Risks and fallbacks

Risk	Fallback
likwid-perfctr can't read counters (permissions, or AMD data-fabric counters missing)	Ask on the CIUK channel on Tuesday. Meanwhile, build the roofline from the theoretical AI plus Grid's reported FLOP/s and the likwid bandwidth, and say clearly which numbers are estimates.
Grid build fails, or pixi has no internet access	Do a manual `configure` with COSMA modules, using the same Grid commit (`e2d607f`). A gives it until Tue 15:00, then asks for help in the team.
Queue contention with other teams	Keep the queue full overnight, use arrays, and run priority 1 first. Never leave nodes idle while we analyse.
1-thread Grid runs exceed the 1-hour limit	Use a smaller `MAX_L` for low thread counts and normalise to the smallest completed thread count.
DINE2 turns out to be a very different node (for example GPU-heavy, with a different CPU)	We use the CPU only. That difference becomes the core of the node-comparison section.
Results become inconsistent between team members	Always submit through `scripts/submit.sh`. Every run directory has `meta.txt` with the git hash, modules and environment. Commit after every batch.

8 Repository conventions

- **Repo layout:**
  - `slurm/`: job scripts. `scripts/submit.sh`: the only way to submit.
  - `results/<node>/<experiment>/<jobid>/`: raw output and metadata for each job.
  - `results/runlog.csv`: one line per job.
  - `analysis/`: `collect.py`, then `plots.py`, writing to `report/figs`.
- **Tag non-default runs.** Anything that isn't a default run gets a `TAG`, such as `TAG=tune-clang-spread`, so it lands in its own experiment directory.
- **Commit results as soon as jobs finish.** Use a commit message like `results: dine likwid threads x3`.
- **Lock any shared COSMA directory** with `setfacl` so other teams can't read it.